CLOUD-NATIVE AI MICROSERVICE

Beginner's guide to RAG, vector search, Docker, Kubernetes and Jenkins CI/CD

Project: Cloud-Native AI Microservice & Automated CI/CD Pipeline

Repository: github.com/ChirayuAgrawal-dev/cloud-native-microservice-pipeline

From a user's question to a context-aware answer — and from a Git commit to a verified local deployment.

Prepared for first-time readers of the repository. This guide explains the purpose of each known file, how the application works, and how the delivery workflow connects the pieces.

Accuracy note: the public repository and local working tree can differ. This guide uses the repository source visible during preparation plus the project's recent reported frontend and deployment workflow. Compare every filename and command against the exact commit you publish.

Contents

1. Project overview
2. Architecture diagrams
3. RAG explained simply
4. Question-answering flow
5. Ingestion and embeddings
6. Technology guide
7. File-by-file guide
8. Docker Compose
9. Kubernetes and Minikube
10. Jenkins CI/CD
11. Configuration and API
12. First-run checklist
13. Troubleshooting
14. Glossary and future improvements

Quick mental model

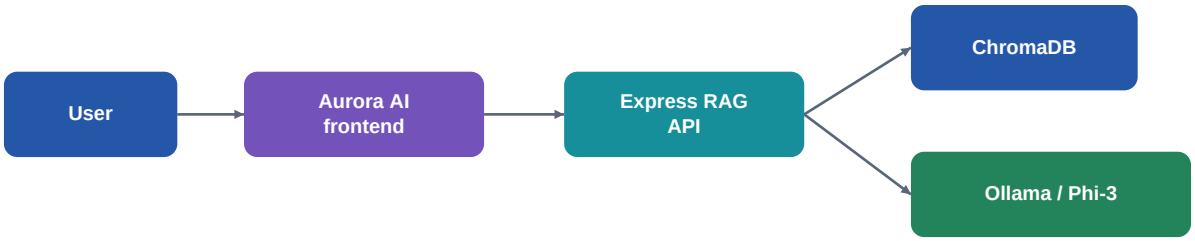
There are two main flows. The RAG flow answers questions by retrieving relevant documents and asking an LLM to generate a response. The CI/CD flow validates source code, builds a container image, and deploys the application. Docker packages; Compose coordinates local containers; Kubernetes manages cluster workloads; Jenkins automates steps.

1. Project overview

This project is a Retrieval-Augmented Generation (RAG) question-answering service. A user submits a question through the Aurora AI web interface or directly to the API. A Node.js/Express backend embeds the question, retrieves the two closest document matches from ChromaDB, combines those snippets with the question, and asks the Phi-3 model served by Ollama to generate an answer.

The operations side demonstrates containerization and delivery automation: Docker images package the code, Docker Compose runs the local services, Minikube runs Kubernetes workloads locally, and Jenkins checks out code, validates files and JavaScript syntax, builds/verifies a Docker image, then invokes a Windows deployment script.

Application request path



Delivery pipeline

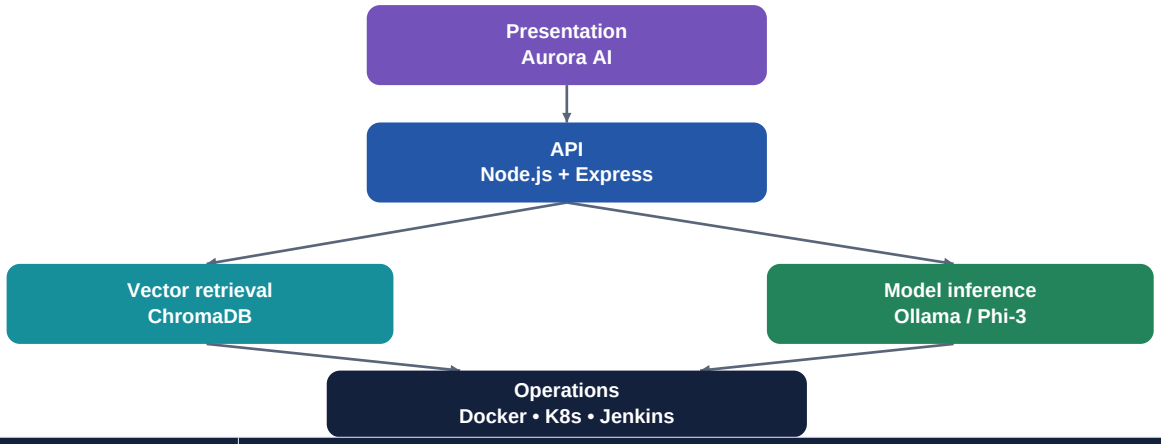


The project is a local cloud-native demonstration, not automatically a production deployment. A green pipeline confirms the configured stages completed, but does not prove security, scale, persistence or model-answer quality.

2. Architecture diagrams

The application has a presentation layer, API layer, retrieval layer, model-serving layer and an operations layer.

Logical architecture



Component	Responsibility
Aurora AI frontend	Collects a question and displays the response.
Express backend	Validates input, retrieves context, calls the model and returns JSON.
ChromaDB	Stores document vectors and retrieves semantically similar text.
Ollama	Hosts the embedding and text-generation model APIs.
Docker / Compose	Packages services and runs the local multi-container setup.
Kubernetes / Minikube	Manages Pods, Deployments and stable Service endpoints.
Jenkins	Automates code checks, image build and deployment steps.

Inside Compose/Kubernetes, services use DNS names such as `ollama` and `chromadb`. `localhost` inside a container refers to that same container, not another service.

3. RAG explained simply

RAG stands for Retrieval-Augmented Generation. The application searches for relevant text first, then supplies it to a language model as context. Think of an open-book exam: find the useful pages, put them next to the question, then write the answer.

RAG = retrieve + augment + generate



Why use RAG?

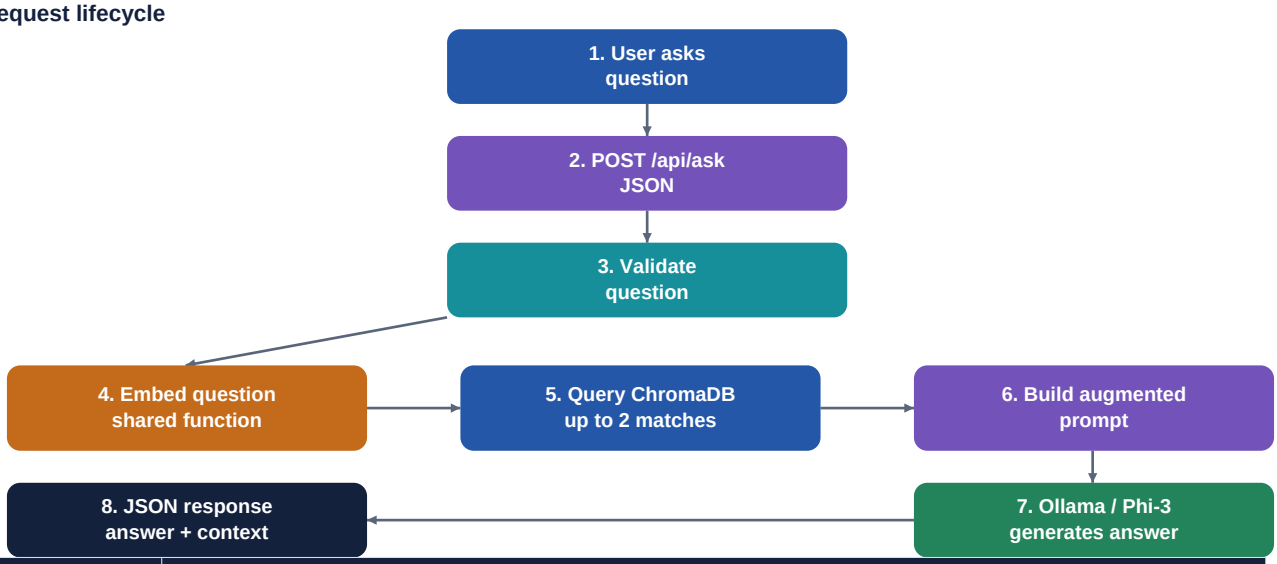
• It lets the app answer using project-specific documents that may not be part of the model's training.
• Retrieval supplies evidence at question time without fine-tuning the model.
• The API returns `contextUsed`, which helps a developer inspect what text was supplied to the model.
• RAG can improve relevance but does not guarantee correctness; the model can still misread context or generate unsupported claims.

Term	Plain-English meaning
Document	A piece of text included in the knowledge base.
Embedding	A list of numbers representing semantic features of text.
Vector database	A database designed to index vectors and associated text.
Semantic search	Finding related meaning, not only exact keyword matches.
Prompt augmentation	Adding retrieved text to the question before generation.

4. Question-answering flow

The main endpoint is `POST /api/ask`. The client sends JSON containing a `question`. The visible `server.js` requests two results from the `rag_documents` collection.

Request lifecycle



Step	What happens
1–2	The frontend sends an HTTP POST with a JSON question.
3	Missing question is rejected with HTTP 400.
4–5	The shared embedding function is used and ChromaDB returns up to two matching documents.
6	Backend constructs a prompt containing context and question.
7	Axios calls Ollama `/api/generate` with model `phi3` and non-streaming response mode.
8	API returns `question`, `contextUsed` and `answer`.

Unexpected failures such as unreachable ChromaDB or Ollama are caught and returned as HTTP 500 with an error/details response. Detailed internal errors are useful during development but should be handled carefully in a public production API.

5. Ingestion and embeddings

Retrieval works only after source text has been embedded and stored. `ingest.js` populates the `rag_documents` collection. It imports the shared embedding function from `embeddings.js`, which is also used when a user asks a question.

Document ingestion flow



An embedding model converts text into a numeric vector. Similar meanings often produce vectors that are close in the embedding space. ChromaDB stores the vectors with document text, then compares a question vector against stored vectors.

Why the same embedding function?

Document and query vectors need to be comparable. Changing the embedding model between ingestion and query can create dimension mismatches or poor retrieval. Sharing `embeddings.js` keeps the process consistent.

Typical first setup

1. Start Ollama and ChromaDB.
2. Pull `phi3` and `nomic-embed-text` into Ollama.
3. Run `npm run ingest` in the backend container.
4. Submit a question to `POST /api/ask`.
5. Inspect `contextUsed` to understand what was retrieved.

Docker Compose defines named volumes for Ollama and ChromaDB. The currently visible Kubernetes manifest uses `emptyDir` volumes for these services, so their data may be lost when Pods are recreated.

6. Technology guide

Technology	Use in this project
JavaScript / Node.js	Runs the API backend and ingestion script.
Express.js	Defines HTTP routes and JSON handling.
Axios	Calls Ollama's HTTP API from the backend.
ChromaDB	Stores embeddings and performs semantic retrieval.
Ollama	Serves models locally behind an HTTP API.
Phi-3 (`phi3`)	Generates a natural-language answer from the augmented prompt.
Nomic Embed Text	Turns documents and questions into embeddings.
Docker	Packages the backend/frontend into container images.
Docker Compose	Starts the local frontend, backend, ChromaDB and Ollama services together.
Kubernetes	Manages container workloads, Pods, Deployments and Services.
Minikube	Runs a local Kubernetes cluster on a developer machine.
Jenkins	Runs repeatable validation, image build and deployment stages.
Git / GitHub	Tracks source changes and provides the remote repository for Jenkins checkout.
Nginx	Serves static frontend files and can proxy API requests as configured in `nginx.conf`.

Cloud-native terminology

The project uses cloud-native patterns locally. Minikube is not a managed cloud Kubernetes service. It is more accurate to describe this as a local containerized application with Kubernetes deployment and CI/CD automation than to claim it is production-grade.

Separate the roles: ChromaDB retrieves context; Ollama runs models; Express coordinates the request; Docker packages code; Kubernetes runs it; Jenkins automates the delivery process.

7. File-by-file guide

The table below covers the known root files and the frontend/supporting files from the recent project work. Compare with your final committed repository tree before relying on this as an exact inventory.

File / folder	Purpose
`server.js`	Main Express API: reads configuration, validates question, opens `rag_documents`, queries two matches, builds prompt, calls Phi-3 through Ollama, returns JSON.
`embeddings.js`	Shared embedding function. Sends text to Ollama using `EMBED_MODEL` (default `nomic-embed-text`) and returns vectors.
`ingest.js`	Loads knowledge documents into ChromaDB with embeddings. Run after services/models are ready and when knowledge content changes.
`package.json`	Project metadata, ES module setting, dependencies (`express`, `axios`, `chromadb`) and scripts (`start`, `ingest`).
`package-lock.json`	Locks the resolved npm dependency tree for more repeatable installs.
`Dockerfile`	Defines how the backend container image is built: base runtime, dependencies and application files.
`docker-compose.yml`	Declares web, Ollama, ChromaDB and frontend services; ports, dependencies and named volumes.
`k8s-deployment.yaml`	Kubernetes Deployments/Services for ChromaDB, Ollama, `ccd-web` and `ccd-frontend`; defines ports and internal/external service types.
`Jenkinsfile`	Pipeline-as-code: checkout, agent verification, npm install, syntax checks, file checks, Docker build/inspect and deployment stage.
`deploy.ps1`	Windows deployment helper called by Jenkins; connects the deployment stage to Docker/Minikube commands. Consult its current source for exact behavior.
`frontend/index.html`	Aurora AI interface markup and client-side interaction.
`frontend/Dockerfile`	Packages the frontend assets into a container served by Nginx.
`frontend/nginx.conf`	Nginx configuration for static files and configured API routing.
`documents/api-documentation.md`	Human-readable description of the API, its request/response contract and usage.
`documents/architecture.txt`	Text notes describing the system architecture.
`README.md`	Repository entry point with overview, setup steps, configuration, deployment instructions and project status.
`LICENSE`	Defines legal terms for reuse of the repository.
`.gitignore` (if present)	Excludes local build output, environment files, secrets and other generated files from Git. Verify it does not allow secrets to be committed.

Dependency map

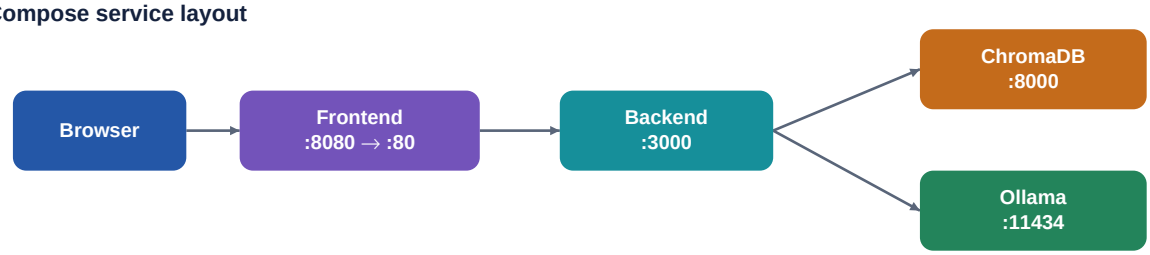
```
server.js ■■imports■■> embeddings.js ■■calls■■> Ollama embedding API
    ■
    ■■■ query ■■> ChromaDB collection: rag_documents
    ■■■ prompt ■■> Ollama /api/generate (phi3)

ingest.js ■■uses same embeddings.js■■> ChromaDB
Jenkinsfile ■■builds Dockerfile image■■> deploy.ps1 ■■updates■■> Minikube
```

8. Docker Compose

Docker Compose reads `docker-compose.yml` and starts the services on a shared network. The backend uses service names such as `ollama` and `chromadb` to call the other containers.

Compose service layout



Service	Host mapping	Purpose
Frontend	8080 → 80	Open Aurora AI at `http://localhost:8080`.
Backend	3000 → 3000	API at `http://localhost:3000/api/ask`.
ChromaDB	8000 → 8000	Vector database API.
Ollama	11434 → 11434	Embedding and generation model API.

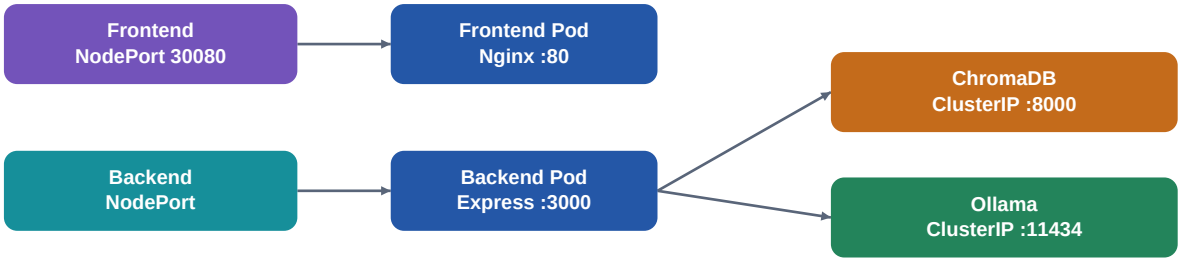
```
docker compose up --build -d
docker compose ps
docker compose logs -f web
docker compose exec ollama ollama pull phi3
docker compose exec ollama ollama pull nomic-embed-text
docker compose exec web npm run ingest
docker compose down
```

Named volumes preserve model and vector data across ordinary container recreation. `docker compose down -v` removes named volumes too; only use it when you intentionally want to delete that stored data.

9. Kubernetes and Minikube

Kubernetes manages container workloads. A Pod runs one or more containers; a Deployment maintains the desired replicas and helps perform rollouts; a Service gives a stable network endpoint to matching Pods. Minikube runs a local cluster for development and demonstrations.

Kubernetes relationships



Service	Type	Observed mapping
`ccd-frontend`	NodePort	80:30080/TCP
`ccd-web`	NodePort	3000:32638/TCP
`chromadb`	ClusterIP	8000/TCP
`ollama`	ClusterIP	11434/TCP

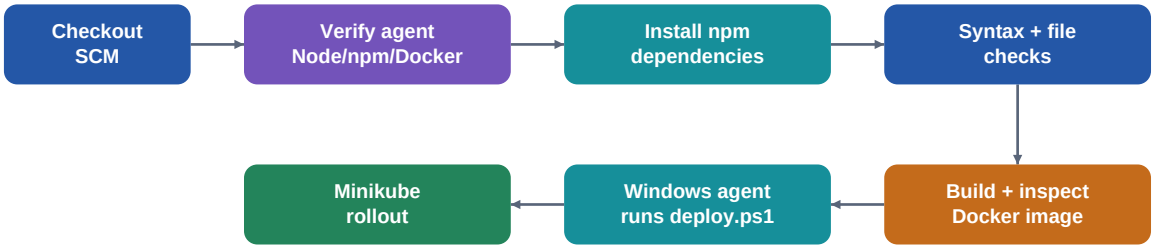
```
minikube status
kubectl get deployments
kubectl get pods -o wide
kubectl get services
kubectl logs deployment/ccd-web
kubectl rollout status deployment/ccd-web
minikube service ccd-frontend --url
minikube service ccd-web --url
```

On Windows with the Docker driver, `minikube service --url` can create a temporary tunnel. Keep that terminal open while using the URL. The visible Kubernetes manifest uses `emptyDir` for ChromaDB and Ollama storage, so data may be lost when those Pods are recreated; persistent volumes are a future improvement.

10. Jenkins CI/CD pipeline

The `Jenkinsfile` defines the pipeline in version control. The main stages use an agent labelled `docker-build`; the deployment stage switches to a Windows agent labelled `windows-deploy` and runs `deploy.ps1`. This separation allows Linux-based checks/build work and Windows-specific Minikube deployment work.

Pipeline stages



Stage	Purpose
Checkout	Fetches the configured repository revision.
Verify Build Agent	Checks Node.js, npm and Docker availability.
Install Dependencies	Runs `npm install --no-audit --no-fund`.
Syntax Check	Runs `node --check` on `server.js`, `ingest.js` and `embeddings.js`.
Verify Project Files	Confirms key files including `package.json`, lockfile, Dockerfile and `deploy.ps1` exist.
Build Docker Image	Builds `ccd-microservice-project:web-ci-test`.
Verify Docker Image	Inspects the built image.
Deploy to Minikube	Checks out code on Windows agent and invokes `deploy.ps1`; a failed script causes the stage to fail.

A successful Jenkins run was reported by the project author. It means the configured stages completed without Jenkins marking the build failed. It does not automatically prove load capacity, security, persistence or that every generated answer is correct.

Windows agent startup: automatic startup after a Windows restart has not been confirmed as configured. Until a startup/reconnection mechanism is implemented and tested, the agent may need to be started manually.

11. Configuration and API contract

Variable	Default / value	Meaning
`PORT`	3000	Port used by Express.
`OLLAMA_HOST`	`http://ollama:11434`	Ollama API base URL.
`CHROMA_HOST`	`http://chromadb:8000`	ChromaDB API base URL.
`EMBED_MODEL`	`nomic-embed-text`	Model used by the embedding function.

Request example

```
POST /api/ask
Content-Type: application/json

{
  "question": "What does this project do?"
}
```

Response shape

```
{
  "question": "What does this project do?",
  "contextUsed": "Relevant retrieved document text...",
  "answer": "Generated answer based on the prompt..."
}
```

A missing question returns HTTP 400. Processing or upstream failures return HTTP 500 with an error/details response. The exact answer depends on the ingested documents and model output.

Container networking reminder

Inside a container, `localhost` points to that container itself. Use Compose/Kubernetes service DNS names to reach sibling services. If the UI opens but the query fails, inspect the frontend proxy configuration, backend logs, model availability, service names and ports.

12. First-run checklist

A. Docker Compose

```
git clone https://github.com/ChirayuAgrawal-dev/cloud-native-microservice-pipeline.git
cd cloud-native-microservice-pipeline
docker compose up --build -d
docker compose ps
docker compose exec ollama ollama pull phi3
docker compose exec ollama ollama pull nomic-embed-text
docker compose exec web npm run ingest

# UI: http://localhost:8080
# API: http://localhost:3000/api/ask
```

B. PowerShell API test

```
$body = @{ question = "What does this project do?" } | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:3000/api/ask" `
  -Method Post -ContentType "application/json" -Body $body
```

C. Minikube

```
minikube start --memory=6144
minikube image build -t ccd-microservice-project-web:latest .
kubectl apply -f k8s-deployment.yaml
kubectl get pods
kubectl get services
kubectl exec -it deploy/ollama -- ollama pull phi3
kubectl exec -it deploy/ollama -- ollama pull nomic-embed-text
kubectl exec -it deploy/ccd-web -- npm run ingest
```

Important: the Jenkinsfile's image tag is `ccd-microservice-project:web-ci-test`, while a manual README command may use `ccd-microservice-project-web:latest`. These are different names. Follow the exact tag expected by the current manifest and deployment script; do not mix tags without updating the configuration.

D. Jenkins

- Confirm Pipeline script from SCM points to this repository and `*/main`.
- Confirm the `docker-build` agent has Node.js, npm and Docker access.
- Confirm the `windows-deploy` agent can access Docker Desktop, Minikube and kubectl.
- Run Build Now and inspect the stage view and Console Output.
- After success, verify Pods and test the UI/API.

13. Troubleshooting

Symptom	What to check
First answer is slow	Model warm-up, CPU/RAM pressure and Ollama logs.
`Cannot GET /` at API root	Expected if no root route exists. Test `POST /api/ask`.
No useful retrieved context	Run ingestion; verify collection name and shared embedding model.
Ollama connection refused	Check container/Pod state, port 11434 and service DNS.
UI works but query fails	Check `nginx.conf`, API routing, backend logs and environment variables.
Minikube URL stops working	Keep the `minikube service --url` terminal open or run it again.
Models/documents disappear in Kubernetes	Manifest uses `emptyDir`; re-pull models and re-ingest after Pod recreation.
Jenkins deployment stage fails	Check Windows agent status, Docker Desktop, Minikube context, kubectl and `deploy.ps1` output.
Agent missing after reboot	Automatic Windows agent startup is not confirmed; manually reconnect until startup is configured.
Git push rejected	Fetch remote changes and resolve conflicts; avoid force-pushing shared `main`.

Avoid these misconceptions

- RAG is not model training. It retrieves existing content at question time.
- ChromaDB does not generate prose; it retrieves context. Ollama's model generates the answer.
- Docker Compose and Kubernetes are different orchestration tools.
- A successful pipeline is not the same as production readiness.
- Local Minikube is not the same as deploying to a managed cloud Kubernetes cluster.

14. Glossary and future improvements

Term	Meaning
API	Defined interface for software components to communicate.
HTTP / REST	Web request-response conventions used by the API.
JSON	Text format used in request and response bodies.
LLM	Large Language Model that generates text from a prompt.
RAG	Retrieve relevant content, then generate an answer using that content.
Embedding	Numeric representation of text used for similarity comparison.
Image	Packaged template used to start a container.
Container	An isolated running process created from an image.
Pod	Kubernetes's smallest deployable unit.
Deployment	Kubernetes resource that maintains replicas and performs rollouts.
Service	Stable network endpoint for a set of Pods.
CI	Continuous Integration: automated checks for code changes.
CD	Continuous Delivery/Deployment: automated release or rollout steps.
Pipeline as code	Build/deploy workflow stored in a version-controlled file.

Reasonable next improvements

• Add API unit/integration tests and retrieval quality checks.
• Document how source files map to the knowledge collection; add metadata/citations where useful.
• Replace temporary Kubernetes `emptyDir` storage with persistent volumes where appropriate.
• Add health/readiness probes, resource limits and structured logs.
• Pin dependencies and use a clear image-tagging strategy.
• Add a tested rollback path and avoid exposing internal error details publicly.
• Configure and test automatic Windows deployment-agent startup after reboot.
• Add monitoring, security checks, load testing and answer-quality evaluation.

Repository

Source repository: github.com/ChirayuAgrawal-dev/cloud-native-microservice-pipeline

This guide is documentation, not a replacement for the source code. Before publishing it, compare the file inventory and commands with the exact repository commit. Keep the README and PDF consistent with the pipeline's actual behavior.